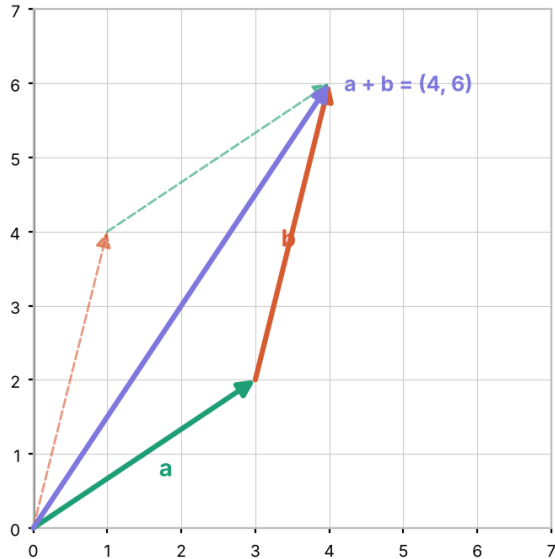


Vector operations

Operacje na wektorach · addition, subtraction, multiplication, division

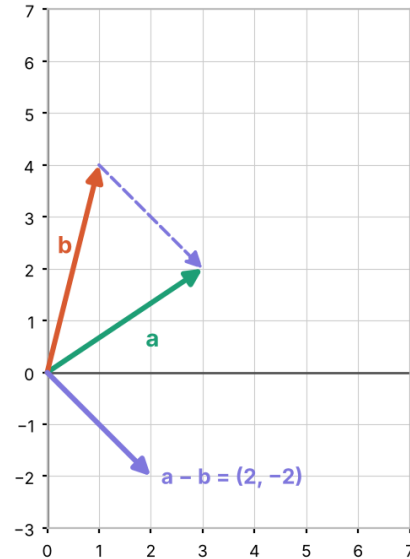
Addition: $a + b$

Tip-to-tail, $a = (3, 2)$, $b = (1, 4)$



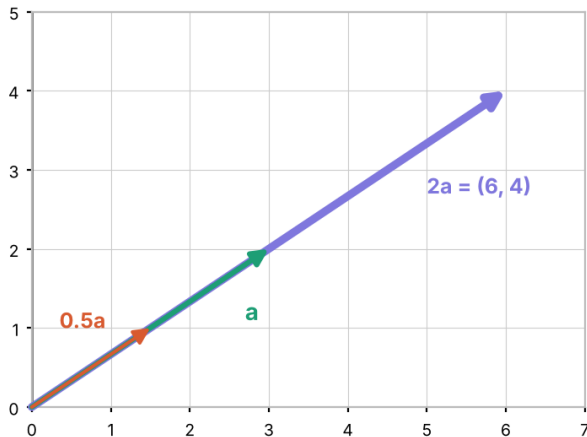
Subtraction: $a - b$

Same vectors, arrow from b to a



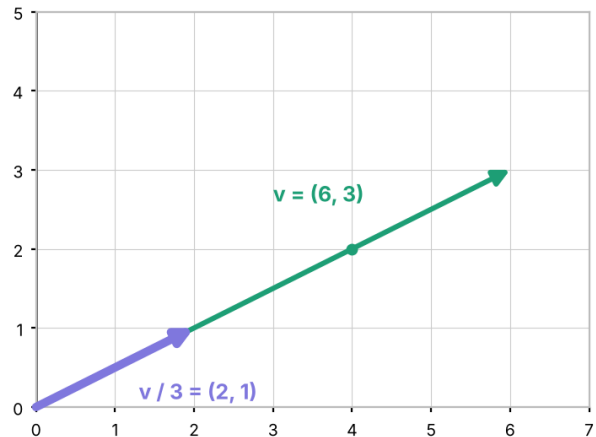
Multiplication: $k \cdot a$

Scalar multiples of $a = (3, 2)$



Division: v / k

$v = (6, 3)$ cut into 3 equal parts



Teal: original vector. Coral: second vector. Purple: result. Dividing by a vector is not defined.

Kolor zielony: wektor wyjściowy. Koralowy: drugi wektor. Fioletowy: wynik. Dzielenie przez wektor nie jest zdefiniowane.

English

A vector has a magnitude and a direction. In components, a 2D vector is $\mathbf{a} = (a_1, a_2)$, and a 3D vector is $\mathbf{a} = (a_1, a_2, a_3)$.

1. Adding and subtracting

Adding vectors: add the matching components.

$$\mathbf{a} + \mathbf{b} = (a_1 + b_1, a_2 + b_2). \text{ Example: } (3, 2) + (1, 4) = (4, 6).$$

Geometrically (tip-to-tail): draw $\mathbf{a}$, then draw $\mathbf{b}$ starting at the tip of $\mathbf{a}$. The sum is the arrow from the start of $\mathbf{a}$ to the tip of $\mathbf{b}$. This is equivalent to the parallelogram rule.

Subtracting vectors: subtract the matching components.

$$\mathbf{a} - \mathbf{b} = (a_1 - b_1, a_2 - b_2). \text{ Example: } (3, 2) - (1, 4) = (2, -2).$$

Geometrically: $\mathbf{a} - \mathbf{b} = \mathbf{a} + (-\mathbf{b})$, where $-\mathbf{b}$ is $\mathbf{b}$ with its direction reversed. It is also the arrow pointing from the tip of $\mathbf{b}$ to the tip of $\mathbf{a}$.

Rules

- Both vectors must have the same number of dimensions.
- Addition is commutative: $\mathbf{a} + \mathbf{b} = \mathbf{b} + \mathbf{a}$.
- Subtraction is not: $\mathbf{a} - \mathbf{b} \neq -(\mathbf{b} - \mathbf{a})$.

2. Multiplying and dividing

Vectors don't have a single "multiply" or "divide" operation. There are several, and **division by a vector is not defined**.

Scalar multiplication (vector $\times$ number): multiply every component. $k \cdot \mathbf{a} = (k \cdot a_1, k \cdot a_2)$. Example: $3 \cdot (2, 1) = (6, 3)$. It stretches the vector ($|k| > 1$) or shrinks it ($|k| < 1$). A negative k also reverses its direction.

Division by a scalar: multiplication by $1/k$ ($k \neq 0$). Example: $(6, 3) / 3 = (2, 1)$. Dividing a vector by its length gives a **unit vector** (length 1): $\hat{\mathbf{a}} = \mathbf{a} / |\mathbf{a}|$. This is normalization in ML.

Dot product (vector $\cdot$ vector $\rightarrow$ a number): multiply matching components and sum. $\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 = |\mathbf{a}| |\mathbf{b}| \cos \theta$. Example: $(3, 2) \cdot (1, 4) = 3 \cdot 1 + 2 \cdot 4 = 11$. If it is 0, the vectors are perpendicular. It is the core operation in linear regression, neural network layers and cosine similarity.

Cross product (only 3D, result is a vector perpendicular to both): $\mathbf{a} \times \mathbf{b} = (a_2 b_3 - a_3 b_2, a_3 b_1 - a_1 b_3, a_1 b_2 - a_2 b_1)$. Rarely needed in ML.

Element-wise (Hadamard) product: multiply matching components, keep a vector. $(3, 2) \circ (1, 4) = (3, 8)$. Element-wise division works the same way: $(3, 2) / (1, 4) = (3, 0.5)$. This is a NumPy operation, not a standard math operation on vectors.

In Python (NumPy)

```
import numpy as np
a = np.array([3, 2])
b = np.array([1, 4])

a + b          # array([4, 6])      addition
a - b          # array([ 2, -2])    subtraction
3 * a          # array([9, 6])      scalar multiplication
a / 3          # array([1., 0.667]) scalar division
np.dot(a, b)   # 11                 dot product
a * b          # array([3, 8])      element-wise product
a / b          # array([3., 0.5])    element-wise division
a / np.linalg.norm(a) # unit vector
np.cross([1,0,0],[0,1,0]) # array([0,0,1]) cross product (3D)
```

Watch out: in NumPy, $\mathbf{a} * \mathbf{b}$ is element-wise, not the dot product. This is a common mistake.

Polski

Wektor ma długość (wartość) i kierunek. W postaci współrzędnych wektor 2D to $\mathbf{a} = (a_1, a_2)$, a wektor 3D to $\mathbf{a} = (a_1, a_2, a_3)$.

1. Dodawanie i odejmowanie

Dodawanie wektorów: dodaj odpowiadające sobie współrzędne.

$$\mathbf{a} + \mathbf{b} = (a_1 + b_1, a_2 + b_2). \text{ Przykład: } (3, 2) + (1, 4) = (4, 6).$$

Geometrycznie (metoda „początek do końca”): narysuj $\mathbf{a}$, potem narysuj $\mathbf{b}$ zaczynając od końca $\mathbf{a}$. Suma to strzałka od początku $\mathbf{a}$ do końca $\mathbf{b}$. Odpowiada to regule równoległoboku.

Odejmowanie wektorów: odejmij odpowiadające sobie współrzędne.

$$\mathbf{a} - \mathbf{b} = (a_1 - b_1, a_2 - b_2). \text{ Przykład: } (3, 2) - (1, 4) = (2, -2).$$

Geometrycznie: $\mathbf{a} - \mathbf{b} = \mathbf{a} + (-\mathbf{b})$, gdzie $-\mathbf{b}$ to wektor $\mathbf{b}$ o odwróconym zwrocie. Jest to też strzałka od końca $\mathbf{b}$ do końca $\mathbf{a}$.

Zasady

- Oba wektory muszą mieć tyle samo wymiarów.
- Dodawanie jest przemienne: $\mathbf{a} + \mathbf{b} = \mathbf{b} + \mathbf{a}$.
- Odejmowanie nie jest: $\mathbf{a} - \mathbf{b} = -(\mathbf{b} - \mathbf{a})$.

2. Mnożenie i dzielenie

Wektory nie mają jednej operacji „mnożenia” ani „dzielenia”. Jest kilka rodzajów mnożenia, a **dzielenie przez wektor nie jest zdefiniowane**.

Mnożenie przez skalar (wektor $\times$ liczba): pomnóż każdą współrzędną. $k \cdot \mathbf{a} = (k \cdot a_1, k \cdot a_2)$. Przykład: $3 \cdot (2, 1) = (6, 3)$. Wektor się wydłuża ($|k| > 1$) lub skraca ($|k| < 1$). Ujemne k dodatkowo odwraca zwrot.

Dzielenie przez skalar: mnożenie przez $1/k$ ($k \neq 0$). Przykład: $(6, 3) / 3 = (2, 1)$. Podzielenie wektora przez jego długość daje **wektor jednostkowy** (długość 1): $\hat{\mathbf{a}} = \mathbf{a} / |\mathbf{a}|$. W ML nazywa się to normalizacją.

Iloczyn skalarny (wektor $\cdot$ wektor $\rightarrow$ liczba): pomnóż odpowiadające sobie współrzędne i zsumuj. $\mathbf{a} \cdot \mathbf{b} = a_1b_1 + a_2b_2 = |\mathbf{a}||\mathbf{b}|\cos \theta$. Przykład: $(3, 2) \cdot (1, 4) = 3 \cdot 1 + 2 \cdot 4 = 11$. Jeśli wynosi 0, wektory są prostopadłe. To podstawowa operacja w regresji liniowej, warstwach sieci neuronowych i podobieństwie cosinusowym.

Iloczyn wektorowy (tylko 3D, wynik to wektor prostopadły do obu): $\mathbf{a} \times \mathbf{b} = (a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1)$. W ML rzadko potrzebny.

Iloczyn po współrzędnych (Hadamarda): pomnóż odpowiadające sobie współrzędne, wynik to nadal wektor. $(3, 2) \circ (1, 4) = (3, 8)$. Dzielenie po współrzędnych działa tak samo: $(3, 2) / (1, 4) = (3, 0.5)$. To operacja z NumPy, a nie standardowa operacja matematyczna na wektorach.

W Pythonie (NumPy)

```
import numpy as np
a = np.array([3, 2])
b = np.array([1, 4])
```

<code>a + b</code>	<code># array([4, 6])</code>	dodawanie
<code>a - b</code>	<code># array([2, -2])</code>	odejmowanie
<code>3 * a</code>	<code># array([9, 6])</code>	mnożenie przez skalar
<code>a / 3</code>	<code># array([1., 0.667])</code>	dzielenie przez skalar
<code>np.dot(a, b)</code>	<code># 11</code>	iloczyn skalarny
<code>a * b</code>	<code># array([3, 8])</code>	iloczyn po współrzędnych
<code>a / b</code>	<code># array([3., 0.5])</code>	dzielenie po współrzędnych
<code>a / np.linalg.norm(a)</code>	<code># wektor jednostkowy</code>	
<code>np.cross([1,0,0],[0,1,0])</code>	<code># array([0,0,1])</code>	iloczyn wektorowy (3D)

Uwaga: w NumPy `a * b` to mnożenie po współrzędnych, a nie iloczyn skalarny. To częsty błąd.